

Paso 1: Crear archivo de práctica inicial

Crear archivo:

```
touch ejercicios/ejercicio_01_tabla_no_normalizada.sql
```

Contenido:

```
DROP TABLE IF EXISTS normalizacion.ventas_no_normalizadas;

CREATE TABLE normalizacion.ventas_no_normalizadas (
    id_venta SERIAL PRIMARY KEY,
    cliente VARCHAR(100),
    correo VARCHAR(120),
    productos TEXT,
    ciudad_cliente VARCHAR(100),
    vendedor VARCHAR(100)
);

INSERT INTO normalizacion.ventas_no_normalizadas
(cliente, correo, productos, ciudad_cliente, vendedor)
VALUES
('Ana Torres', 'ana@mail.com', 'Mouse, Teclado, Monitor', 'Bogotá',
'Carlos'),
('Luis Gómez', 'luis@mail.com', 'Laptop, Mouse', 'Medellín', 'Carlos'),
('Ana Torres', 'ana@mail.com', 'Silla', 'Bogotá', 'Diana');

SELECT * FROM normalizacion.ventas_no_normalizadas;
```

Ejecutar desde terminal:

```
docker exec -i postgres16_bdcurso psql -U curso_user -d curso_bd <
ejercicios/ejercicio_01_tabla_no_normalizada.sql
```

Paso 2: Analizar los errores de diseño

Preguntas para el aprendiz:

1. ¿Qué datos se repiten?
2. ¿Qué columna tiene varios valores en una sola celda?
3. ¿Qué pasa si un cliente cambia de correo?
4. ¿Qué pasa si se elimina la única venta de un cliente?
5. ¿Se puede registrar un producto sin venta?
6. ¿Se puede registrar un vendedor sin venta?
7. ¿Qué entidades reales aparecen en la tabla?

Entidades esperadas:

- Cliente.
 - Producto.
 - Vendedor.
 - Venta.
 - Detalle de venta.
-

Paso 3: Llevar la tabla a Primera Forma Normal

Crear archivo:

```
touch ejercicios/ejercicio_02_primera_forma_normal.sql
```

Contenido:

```
DROP TABLE IF EXISTS normalizacion.ventas_1fn;

CREATE TABLE normalizacion.ventas_1fn (
    id_venta INT,
    cliente VARCHAR(100),
    correo VARCHAR(120),
    producto VARCHAR(100),
    ciudad_cliente VARCHAR(100),
    vendedor VARCHAR(100)
);

INSERT INTO normalizacion.ventas_1fn
(id_venta, cliente, correo, producto, ciudad_cliente, vendedor)
VALUES
(1, 'Ana Torres', 'ana@mail.com', 'Mouse', 'Bogotá', 'Carlos'),
(1, 'Ana Torres', 'ana@mail.com', 'Teclado', 'Bogotá', 'Carlos'),
(1, 'Ana Torres', 'ana@mail.com', 'Monitor', 'Bogotá', 'Carlos'),
(2, 'Luis Gómez', 'luis@mail.com', 'Laptop', 'Medellín', 'Carlos'),
(2, 'Luis Gómez', 'luis@mail.com', 'Mouse', 'Medellín', 'Carlos'),
(3, 'Ana Torres', 'ana@mail.com', 'Silla', 'Bogotá', 'Diana');

SELECT * FROM normalizacion.ventas_1fn;
```

Ejecutar:

```
docker exec -i postgres16_bdcurso psql -U curso_user -d curso_bd <
ejercicios/ejercicio_02_primera_forma_normal.sql
```

Paso 4: Identificar dependencias funcionales

El aprendiz debe completar una tabla como esta:

Dato	Depende de
cliente	id_venta
correo	cliente
ciudad_cliente	cliente
vendedor	id_venta
producto	producto
cantidad	id_venta + producto

Ejemplo de análisis:

Una venta pertenece a un cliente.
Una venta es atendida por un vendedor.
Un cliente tiene correo y ciudad.
Un producto puede estar en muchas ventas.
Una venta puede tener muchos productos.

Paso 5: Llevar el modelo a Segunda Forma Normal

Crear archivo:

```
touch ejercicios/ejercicio_03_segunda_forma_normal.sql
```

Contenido:

```
DROP TABLE IF EXISTS normalizacion.detalle_ventas CASCADE;  
DROP TABLE IF EXISTS normalizacion.ventas CASCADE;  
DROP TABLE IF EXISTS normalizacion.productos CASCADE;  
DROP TABLE IF EXISTS normalizacion.vendedores CASCADE;  
DROP TABLE IF EXISTS normalizacion.clientes CASCADE;
```

```
CREATE TABLE normalizacion.clientes (  
    id_cliente SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    correo VARCHAR(120) UNIQUE NOT NULL,  
    ciudad VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE normalizacion.vendedores (  
    id_vendedor SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL UNIQUE  
);
```

```
CREATE TABLE normalizacion.productos (  
    id_producto SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    descripcion VARCHAR(255) NOT NULL,  
    precio DECIMAL(10,2) NOT NULL  
);
```

```

        id_producto SERIAL PRIMARY KEY,
        nombre VARCHAR(100) NOT NULL UNIQUE
    );

CREATE TABLE normalizacion.ventas (
    id_venta SERIAL PRIMARY KEY,
    id_cliente INT NOT NULL,
    id_vendedor INT NOT NULL,
    fecha_venta DATE NOT NULL DEFAULT CURRENT_DATE,

    CONSTRAINT fk_ventas_clientes
        FOREIGN KEY (id_cliente)
        REFERENCES normalizacion.clientes(id_cliente),

    CONSTRAINT fk_ventas_vendedores
        FOREIGN KEY (id_vendedor)
        REFERENCES normalizacion.vendedores(id_vendedor)
);

CREATE TABLE normalizacion.detalle_ventas (
    id_venta INT NOT NULL,
    id_producto INT NOT NULL,
    cantidad INT NOT NULL CHECK (cantidad > 0),

    PRIMARY KEY (id_venta, id_producto),

    CONSTRAINT fk_detalle_ventas
        FOREIGN KEY (id_venta)
        REFERENCES normalizacion.ventas(id_venta)
        ON DELETE CASCADE,

    CONSTRAINT fk_detalle_productos
        FOREIGN KEY (id_producto)
        REFERENCES normalizacion.productos(id_producto)
);

```

Ejecutar:

```

docker exec -i postgres16_bdcursos psql -U curso_user -d curso_bd <
ejercicios/ejercicio_03_segunda_forma_normal.sql

```

Paso 6: Insertar datos en el modelo normalizado

Crear archivo:

```
touch ejercicios/insertar_datos_2fn.sql
```

Contenido:

```

INSERT INTO normalizacion.clientes (nombre, correo, ciudad)
VALUES
('Ana Torres', 'ana@mail.com', 'Bogotá'),
('Luis Gómez', 'luis@mail.com', 'Medellín');

INSERT INTO normalizacion.vendedores (nombre)
VALUES
('Carlos'),
('Diana');

INSERT INTO normalizacion.productos (nombre)
VALUES
('Mouse'),
('Teclado'),
('Monitor'),
('Laptop'),
('Silla');

INSERT INTO normalizacion.ventas (id_cliente, id_vendedor, fecha_venta)
VALUES
(1, 1, '2026-06-01'),
(2, 1, '2026-06-02'),
(1, 2, '2026-06-03');

INSERT INTO normalizacion.detalle_ventas (id_venta, id_producto, cantidad)
VALUES
(1, 1, 1),
(1, 2, 1),
(1, 3, 1),
(2, 4, 1),
(2, 1, 1),
(3, 5, 1);

```

Ejecutar:

```

docker exec -i postgres16_bdcurso psql -U curso_user -d curso_bd <
ejercicios/insertar_datos_2fn.sql

```

Paso 7: Validar el modelo con consultas

Crear archivo:

```
touch ejercicios/consultas_validacion_2fn.sql
```

Contenido:

```

SELECT
    v.id_venta,
    c.nombre AS cliente,
    c.correo,
    vd.nombre AS vendedor,
    p.nombre AS producto,

```

```
        dv.cantidad,  
        v.fecha_venta  
FROM normalizacion.ventas v  
JOIN normalizacion.clientes c  
    ON v.id_cliente = c.id_cliente  
JOIN normalizacion.vendedores vd  
    ON v.id_vendedor = vd.id_vendedor  
JOIN normalizacion.detalle_ventas dv  
    ON v.id_venta = dv.id_venta  
JOIN normalizacion.productos p  
    ON dv.id_producto = p.id_producto  
ORDER BY v.id_venta;
```

Ejecutar:

```
docker exec -i postgres16_bdc curso psql -U curso_user -d curso_bd <  
ejercicios/consultas_validacion_2fn.sql
```

Paso 8: Llevar el modelo a Tercera Forma Normal

El problema pendiente está en la tabla `clientes`:

```
clientes(id_cliente, nombre, correo, ciudad)
```

La ciudad puede tener más información asociada:

- Nombre de ciudad.
- Departamento.
- País.

Si dejamos todo en `clientes`, se repetirá información.

Crear archivo:

```
touch ejercicios/ejercicio_04_tercera_forma_normal.sql
```

Contenido:

```
DROP TABLE IF EXISTS normalizacion.detalle_ventas_3fn CASCADE;  
DROP TABLE IF EXISTS normalizacion.ventas_3fn CASCADE;  
DROP TABLE IF EXISTS normalizacion.clientes_3fn CASCADE;  
DROP TABLE IF EXISTS normalizacion.ciudades CASCADE;  
DROP TABLE IF EXISTS normalizacion.departamentos CASCADE;  
DROP TABLE IF EXISTS normalizacion.productos_3fn CASCADE;
```

```

DROP TABLE IF EXISTS normalizacion.vendedores_3fn CASCADE;

CREATE TABLE normalizacion.departamentos (
    id_departamento SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE normalizacion.ciudades (
    id_ciudad SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    id_departamento INT NOT NULL,

    CONSTRAINT fk_ciudad_departamento
        FOREIGN KEY (id_departamento)
        REFERENCES normalizacion.departamentos(id_departamento),

    CONSTRAINT uq_ciudad_departamento
        UNIQUE (nombre, id_departamento)
);

CREATE TABLE normalizacion.clientes_3fn (
    id_cliente SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(120) UNIQUE NOT NULL,
    id_ciudad INT NOT NULL,

    CONSTRAINT fk_cliente_ciudad
        FOREIGN KEY (id_ciudad)
        REFERENCES normalizacion.ciudades(id_ciudad)
);

CREATE TABLE normalizacion.vendedores_3fn (
    id_vendedor SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE normalizacion.productos_3fn (
    id_producto SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE normalizacion.ventas_3fn (
    id_venta SERIAL PRIMARY KEY,
    id_cliente INT NOT NULL,
    id_vendedor INT NOT NULL,
    fecha_venta DATE NOT NULL DEFAULT CURRENT_DATE,

    CONSTRAINT fk_ventas_clientes_3fn
        FOREIGN KEY (id_cliente)
        REFERENCES normalizacion.clientes_3fn(id_cliente),

    CONSTRAINT fk_ventas_vendedores_3fn
        FOREIGN KEY (id_vendedor)
        REFERENCES normalizacion.vendedores_3fn(id_vendedor)
);

CREATE TABLE normalizacion.detalle_ventas_3fn (

```

```

    id_venta INT NOT NULL,
    id_producto INT NOT NULL,
    cantidad INT NOT NULL CHECK (cantidad > 0),

    PRIMARY KEY (id_venta, id_producto),

    CONSTRAINT fk_detalle_ventas_3fn
        FOREIGN KEY (id_venta)
        REFERENCES normalizacion.ventas_3fn(id_venta)
        ON DELETE CASCADE,

    CONSTRAINT fk_detalle_productos_3fn
        FOREIGN KEY (id_producto)
        REFERENCES normalizacion.productos_3fn(id_producto)
);

```

Paso 9: Insertar datos en el modelo 3FN

```

INSERT INTO normalizacion.departamentos (nombre)
VALUES
('Cundinamarca'),
('Antioquia');

INSERT INTO normalizacion.ciudades (nombre, id_departamento)
VALUES
('Bogotá', 1),
('Medellín', 2);

INSERT INTO normalizacion.clientes_3fn (nombre, correo, id_ciudad)
VALUES
('Ana Torres', 'ana@mail.com', 1),
('Luis Gómez', 'luis@mail.com', 2);

INSERT INTO normalizacion.vendedores_3fn (nombre)
VALUES
('Carlos'),
('Diana');

INSERT INTO normalizacion.productos_3fn (nombre)
VALUES
('Mouse'),
('Teclado'),
('Monitor'),
('Laptop'),
('Silla');

INSERT INTO normalizacion.ventas_3fn (id_cliente, id_vendedor, fecha_venta)
VALUES
(1, 1, '2026-06-01'),
(2, 1, '2026-06-02'),
(1, 2, '2026-06-03');

INSERT INTO normalizacion.detalle_ventas_3fn (id_venta, id_producto,
cantidad)

```



```
VALUES
(1, 1, 1),
(1, 2, 1),
(1, 3, 1),
(2, 4, 1),
(2, 1, 1),
(3, 5, 1);
```

Paso 10: Consultar el modelo 3FN

```
SELECT
    v.id_venta,
    c.nombre AS cliente,
    c.correo,
    ci.nombre AS ciudad,
    d.nombre AS departamento,
    ven.nombre AS vendedor,
    p.nombre AS producto,
    dv.cantidad,
    v.fecha_venta
FROM normalizacion.ventas_3fn v
JOIN normalizacion.clientes_3fn c
    ON v.id_cliente = c.id_cliente
JOIN normalizacion.ciudades ci
    ON c.id_ciudad = ci.id_ciudad
JOIN normalizacion.departamentos d
    ON ci.id_departamento = d.id_departamento
JOIN normalizacion.vendedores_3fn ven
    ON v.id_vendedor = ven.id_vendedor
JOIN normalizacion.detalle_ventas_3fn dv
    ON v.id_venta = dv.id_venta
JOIN normalizacion.productos_3fn p
    ON dv.id_producto = p.id_producto
ORDER BY v.id_venta;
```